

Nhập môn quy hoạch động chữ số

Lazycal

Nguồn gốc: First look at digit-based counting problems

Dynamic Programming / Introductory digit DP.pdf

Abstract

Tài liệu nguồn là một bộ slide ngắn về các bài toán đếm theo chữ số. Bản dịch chuyển nội dung thành ghi chú liền mạch: trước hết thống nhất ký hiệu khoảng, sau đó trình bày cách tiền xử lý trạng thái theo số chữ số và cách quét từng chữ số của cận trên để tính số lượng trong một khoảng.

Contents

1 Kiến thức cơ bản	1
2 Tư tưởng và phương pháp cơ bản	1
3 HDU 2089	2
3.1 Tiền xử lý	2
3.2 Đếm tiền tố	3
4 HDU 3652	3
5 URAL 1057	4
6 Một ví dụ với phép xor	4
7 Mở rộng: SPOJ Sorted Bit Sequence	5
8 Kết luận	5

1 Kiến thức cơ bản

Một số ký hiệu khoảng được dùng xuyên suốt tài liệu:

$$[l, r] = \{x \mid l \leq x \leq r\}, \quad [l, r) = \{x \mid l \leq x < r\},$$

$$(l, r] = \{x \mid l < x \leq r\}, \quad (l, r) = \{x \mid l < x < r\}.$$

Dấu ngoặc vuông nghĩa là lấy cả đầu mút; dấu ngoặc tròn nghĩa là không lấy đầu mút đó.

Trong các kỳ thi OI, ta thường gặp yêu cầu đếm số lượng số nguyên thỏa mãn một điều kiện nào đó trong đoạn $[l, r]$. Dạng này thường được đổi thành hiệu của hai truy vấn tiền tố:

$$\text{count}([l, r]) = \text{count}([0, r]) - \text{count}([0, l)).$$

Vì vậy phần chính là xây dựng một hàm đếm các số trong $[0, n)$ hoặc $[0, n]$.

2 Tư tưởng và phương pháp cơ bản

Với một số $x < n$, khi so sánh từ chữ số cao xuống chữ số thấp, luôn tồn tại một vị trí đầu tiên tại đó chữ số của x nhỏ hơn chữ số tương ứng của n . Ví dụ với $n = 58$ trong hệ thập phân:

$$x = 49$$

nhỏ hơn n ngay ở hàng chục, còn

$$x = 51$$

giống n ở hàng chục và nhỏ hơn n ở hàng đơn vị.

Từ nhận xét này, ta quét các chữ số của n từ cao xuống thấp, thử chọn chữ số đầu tiên nhỏ hơn cận trên. Một khi vị trí đó đã được chọn, các chữ số phía sau không còn bị ràng buộc bởi n nữa; chúng có thể chạy từ 00...0 đến 99...9 trong hệ thập phân. Vì phần đuôi đã tự do, ta tiền xử lý số cách cho từng độ dài và từng trạng thái rồi cộng trực tiếp.

Khuôn mẫu trạng thái thường có dạng

$$f[i, \text{st}],$$

trong đó i là số chữ số còn lại cần điền, còn st là trạng thái phụ thuộc vào bài toán. Các số có thể có chữ số 0 ở đầu; chẳng hạn 00058 vẫn được xem là một số có 5 chữ số trong bảng tiền xử lý.

Nếu $i = 4$, thì $f[i, \text{st}]$ đếm các số từ 0000 đến 9999 thỏa điều kiện tương ứng với trạng thái st . Khi quyết định chữ số thứ i , ta thử các giá trị có thể, cập nhật trạng thái mới st' , rồi cộng:

$$f[i, \text{st}] += f[i - 1, \text{st}'].$$

3 HDU 2089

Đề bài yêu cầu đếm trong đoạn $[n, m]$ số lượng số không chứa chuỗi chữ số 62 và cũng không chứa chữ số 4. Ví dụ, 62315 chứa 62, còn 88914 chứa 4, nên cả hai đều bị loại. Ràng buộc trong slide là

$$0 < n \leq m < 1000000.$$

Theo phương pháp chung, ta tiền xử lý bảng f , sau đó trả lời bằng

$$\text{solve}(m) - \text{solve}(n - 1),$$

trong đó $\text{solve}(u)$ đếm các số hợp lệ trong $[0, u]$.

3.1 Tiền xử lý

Đặt $f[i, j]$ là số lượng số có đúng i chữ số, chữ số đầu là j , và không chứa 62 hay 4. Ví dụ $f[2, 6]$ đếm các số hai chữ số bắt đầu bằng 6:

$$60, 61, 63, 65, 66, 67, 68, 69.$$

Hai lựa chọn 62 và 64 bị loại, lần lượt vì sinh chuỗi 62 và vì có chữ số 4.

Khởi tạo

$$f[0, 0] = 1.$$

Với $i = 1, \dots, 7$, thử chữ số đầu j và chữ số kế tiếp k :

$$f[i, j] += f[i - 1, k]$$

nếu $j \neq 4$ và không đồng thời có $j = 6, k = 2$. Dạng vòng lặp là:

```
for i = 1 .. 7
  for j = 0 .. 9
    for k = 0 .. 9
      if j != 4 and not (j == 6 and k == 2)
        f[i][j] += f[i - 1][k]
```

3.2 Đếm tiền tố

Giả sử các chữ số của u được lưu từ phải sang trái:

$\text{digit}[1]$ là hàng đơn vị, $\text{digit}[\text{len}]$ là chữ số cao nhất.

Ví dụ $u = 58$ thì $\text{digit}[1] = 8$ và $\text{digit}[2] = 5$.

Khi quét từ $i = \text{len}$ xuống 1, ta thử mọi chữ số j nhỏ hơn $\text{digit}[i]$ ở vị trí hiện tại. Nếu chọn j hợp lệ với tiền tố đã có, toàn bộ $i - 1$ chữ số phía sau được đếm bằng bảng $f[i, j]$. Nếu chính chữ số của u đã tạo ra chữ số 4 hoặc chuỗi 62, quá trình dừng vì mọi số đi tiếp theo cùng tiền tố đều không hợp lệ.

```
for i = len .. 1
  for j = 0 .. digit[i] - 1
    if j != 4 and not (j == 2 and digit[i + 1] == 6)
      ans += f[i][j]
  if digit[i] == 4 or (digit[i] == 2 and digit[i + 1] == 6)
    break
```

4 HDU 3652

Đề bài yêu cầu đếm các số nhỏ hơn n , chia hết cho 13, và biểu diễn thập phân có chứa chuỗi 13.

Ta vẫn tiền xử lý trước, rồi quét theo chữ số để thống kê tiền tố. Vì bài toán đòi hỏi đồng thời “đã chứa 13 hay chưa” và phần dư modulo 13, ta dùng trạng thái:

$$f[i, j, k, l].$$

Ý nghĩa là: xét các số có i chữ số, chữ số đầu là j ; biến k cho biết đã chứa chuỗi 13 hay chưa; biến l là phần dư modulo 13.

Khi quyết định chữ số thứ i , thử $x = 0, \dots, 9$. Nếu đã chứa 13 thì phần đuôi có thể ở trạng thái đã chứa 13; nếu chưa, cặp chữ số $j = 1, x = 3$ sẽ chuyển trạng thái sang đã chứa. Phần dư được cập nhật bằng giá trị của chữ số mới ở hàng 10^{i-1} :

$$l' \equiv l - j \cdot 10^{i-1} \pmod{13}.$$

Các chuyển trong slide có dạng:

$$\begin{aligned} f[i, j, 1, l] &+= f[i - 1, x, 1, l'], \\ f[i, j, 1, l] &+= f[i - 1, x, 0, l'] \quad \text{nếu } j = 1, x = 3, \\ f[i, j, 0, l] &+= f[i - 1, x, 0, l'] \quad \text{nếu } (j, x) \neq (1, 3). \end{aligned}$$

Khi đếm các số nhỏ hơn n , ta quét từng chữ số giống bài trước. Cần ghi nhớ phần dư của tiền tố đã chọn và chờ cho biết tiền tố đã xuất hiện 13 hay chưa. Với mỗi chữ số thử nhỏ hơn cận, bảng f trả lời số lượng phần đuôi khi phần dư cuối cùng phải bằng 0 modulo 13.

Một dạng mã của bước đếm trong slide:

```
bit[0] = 1;
for (ll i = 1; i <= 12; ++i) bit[i] = bit[i - 1] * 10;

for (ll i = digit[0], mod = 0; i; --i) {
    for (ll j = 0; j < digit[i]; ++j) {
        ans += f[i][j][1][(13 - mod * bit[i] % 13) % 13];
        if (t || (j == 3 && digit[i + 1] == 1))
            ans += f[i][j][0][(13 - mod * bit[i] % 13) % 13];
    }
    if (digit[i + 1] == 1 && digit[i] == 3) t = 1;
    mod = (mod * 10 + digit[i]) % 13;
}
```

5 URAL 1057

Đề bài cho đoạn $[X, Y]$ và yêu cầu đếm số nguyên thỏa mãn: số đó bằng tổng của đúng K lũy thừa nguyên không âm khác nhau của B . Ví dụ với $X = 15, Y = 20, K = 2, B = 2$, ba số thỏa mãn là

$$17 = 2^4 + 2^0, \quad 18 = 2^4 + 2^1, \quad 20 = 2^4 + 2^2.$$

Ràng buộc:

$$1 \leq X \leq Y \leq 2^{31} - 1, \quad 1 \leq K \leq 20, \quad 2 \leq B \leq 10.$$

Một số là tổng của các lũy thừa khác nhau của B khi và chỉ khi trong biểu diễn cơ số B , mỗi chữ số chỉ là 0 hoặc 1, và có đúng K chữ số bằng 1. Vì vậy ta có thể chuyển mọi cơ số B về cùng một bài toán nhị phân trên các chữ số 0, 1.

Đặt

$$\text{cnt}[i, j]$$

là số lượng nhị phân dài i có đúng j bit bằng 1. Tiền xử lý:

$$\text{cnt}[i, j] = \text{cnt}[i - 1, j] + \text{cnt}[i - 1, j - 1].$$

Để tính $\text{count}([0, n])$, đổi n sang cơ số B rồi quét từ trái sang phải. Mỗi khi gặp chữ số 1, ta có thể đặt vị trí hiện tại thành 0 và cộng số cách điền phần sau với đúng số bit 1 còn thiếu:

$$\text{nếu } \text{digit}[i] = 1 \text{ thì } \text{ans} += \text{cnt}[i, \text{need}].$$

Ở đây need là số chữ số 1 còn cần ở phần đuôi. Nếu gặp chữ số lớn hơn 1, ta chọn chữ số hiện tại là 1 rồi đặt mọi chữ số phía sau tự do trong $\{0, 1\}$; sau đó không cần tiếp tục bám sát cận, vì tiền tố đã nhỏ hơn n .

Vấn đề cuối cùng là xử lý cơ số khác 2. Với một truy vấn n , ta cần tìm số lớn nhất không vượt quá n mà biểu diễn cơ số B chỉ chứa 0 và 1. Cách làm là tìm chữ số đầu tiên từ trái sang phải khác 0, 1, đổi nó thành 1 và đặt toàn bộ chữ số bên phải thành 1. Biểu diễn cơ số B thu được có thể được xem như một xâu nhị phân để trả lời truy vấn. Ví dụ:

$$n = (10204)_9 \implies n' = (10111)_2$$

trong không gian đếm các chữ số hợp lệ.

6 Một ví dụ với phép xor

Bài test-09-07-p1 trong slide phát biểu: cho dãy $A[i]$ độ dài n , tính tổng các giá trị

$$A[i] \text{ xor } A[j] \quad (i < j).$$

Ý tưởng cũng là xử lý từng bit. Với mỗi bit, khi xét một phần tử, ta biết trước đó có bao nhiêu phần tử có bit bằng 0 và bao nhiêu phần tử có bit bằng 1; bit hiện tại đóng góp vào tổng đúng khi hai bit khác nhau.

7 Mở rộng: SPOJ Sorted Bit Sequence

Slide gợi ý bài SORTBIT:

<http://www.spoj.pl/problems/SORTBIT>

hoặc bản trên HUST:

<http://acm.hust.edu.cn/vjudge/problem/viewProblem.action?id=18852>.

Nội dung và phân tích của bài này được dẫn tới bài viết tham khảo trong phần cuối.

8 Kết luận

Cốt lõi của các bài toán chữ số là tư tưởng “xác định từng vị trí”. Vì số lượng thao tác cơ bản chỉ ở mức $O(\log n)$ theo giá trị cận trên, ta có thể chấp nhận trạng thái phụ phức tạp hơn một chút, hoặc dùng chia đôi, liệt kê và các kỹ thuật con khác để giảm độ phức tạp tư duy và cài đặt.

Khi cần đếm trong đoạn $[l, r]$, dạng thường dùng vẫn là:

$$\text{count}([l, r]) = \text{count}([0, r]) - \text{count}([0, l)).$$

Tài liệu tham khảo

- Lưu Thông, “Bàn về các bài toán thống kê theo chữ số”, trong tuyển tập thuật toán được dẫn bởi slide.
- <http://hi.baidu.com/billdu/item/c749952ab2ab50c2ef10f137>